

Python の基本的な使用方法

目次

1. Pythonとは
2. Pythonの実行方法
3. プログラムの基本構造
4. 四則計算
5. 変数の使い方
6. リストとディクショナリ
7. 制御構造
8. 文字列処理と正規表現

Pythonとは

始めるには最適なプログラミング言語

- C言語やFortranと比べて学びやすい
- 豊富なライブラリ（補助ツール）
- 計算速度も十分に高速

Pythonの特徴

なぜPythonが人気なのか？

1. 学習のしやすさ

- シンプルで読みやすい構文
- 初心者にも優しい

2. 豊富なライブラリ

- データ分析、機械学習、Web開発など
- 様々な分野で活用可能

3. 活発なコミュニティ

- 豊富なドキュメント
- 問題解決の情報が豊富

Pythonの実行方法

通常の実行手順

1. テキストファイルにプログラムを書く（ソースコード）
2. 拡張子「.py」で保存
3. コマンドラインから実行

```
python your_script.py
```

プログラムの基本構造

Hello Worldプログラム

```
def main():  
    print("Hello world")  
  
if __name__ == "__main__":  
    main()
```

重要な要素：

- `def main():` - メイン関数の定義
- インデント（タブ）でコードブロックを表現
- `if __name__ == "__main__":` - スクリプト実行時のみ実行
 - `python your_script.py` として実行された場合に `main()` を呼び出す

コーディング規約

PEP8とは

- Python Enhancement Proposal 8
- Pythonの統一的なコーディングルール
- 可読性の高いコードを書くための指針

参考： <https://pep8-jp.readthedocs.io/ja/latest/>

ポイント：

- 一貫性のあるインデント
- 適切な空白の使用
- わかりやすい変数名

四則計算

基本的な演算

```
def main():  
    print(3 + 15)    # 加算: 18  
    print(10 - 2)    # 減算: 8  
    print(90 / 3)    # 除算: 30.0  
    print(51 * 5)    # 乗算: 255
```

演算子：

- $+$ 加算
- $-$ 減算
- $/$ 除算
- $*$ 乗算

変数の使い方

変数とは

数値や文字列などのデータを保存する箱

```
def main():  
    a = 3          # 変数aに3を代入  
    b = 18         # 変数bに18を代入  
  
    print(a + b)   # 21  
    print(a - b)   # -15  
    print(b / a)   # 6.0  
    print(a * b)   # 54
```

重要： `=` は「等しい」ではなく「代入」を意味する

変数の命名規則

わかりやすい変数名を使おう

良い例

```
student_count = 30  
average_score = 85.5  
user_name = "太郎"
```

避けるべき例

```
a = 30  
x = 85.5  
s = "太郎"
```

ポイント：

- 意味のある名前をつける
- スネークケース（単語を_で区切る）を使用
- 日本語も使用可能だが英語が推奨

データ型

主なデータ型

```
# 整数 (int)
```

```
age = 25
```

```
# 浮動小数点数 (float)
```

```
temperature = 36.5
```

```
# 文字列 (str)
```

```
name = "Python"
```

```
# 真偽値 (bool)
```

```
is_student = True
```


リスト

複数のデータをまとめて管理

```
def main():  
    # リストの作成  
    numbers = [1, 2, 3, 4, 5]  
    fruits = ["apple", "banana", "orange"]  
  
    # 要素へのアクセス (インデックスは0から)  
    print(fruits[0]) # apple  
    print(fruits[1]) # banana  
  
    # リストへの要素追加  
    fruits.append("grape")  
    print(fruits) # ['apple', 'banana', 'orange', 'grape']
```

リストの操作

よく使う操作

```
numbers = [1, 2, 3, 4, 5]
# 長さを取得
print(len(numbers))    # 5
# スライス（範囲指定）
print(numbers[1:3])    # [2, 3]
# 要素の存在確認
print(3 in numbers)    # True
# ループ処理
for num in numbers:
    print(num)
```

ディクショナリ

キーと値のペアでデータを管理

```
# ディクショナリの作成
student = {
    "name": "太郎",
    "age": 20,
    "grade": "A"
}
# 値の取得
print(student["name"]) # 太郎
print(student["age"])  # 20
# 値の追加・更新
student["email"] = "taro@example.com"
print(student)
```

制御構造：if文

条件分岐

```
score = 85
if score >= 90:
    print("優秀です")
elif score >= 70:
    print("良好です")
elif score >= 60:
    print("合格です")
else:
    print("不合格です")
```

比較演算子：

- `==` 等しい
- `!=` 等しくない
- `>`, `<`, `>=`, `<=` 大小比較

制御構造：for文

繰り返し処理

```
# リストの各要素を処理
fruits = ["apple", "banana", "orange"]
for fruit in fruits:
    print(fruit)
```

```
# 数値の範囲で繰り返し
for i in range(5):
    print(i) # 0, 1, 2, 3, 4
```

```
# インデックス付きで処理
for i, fruit in enumerate(fruits):
    print(f"{i}: {fruit}")
```

制御構造：while文

条件が真の間繰り返す

```
count = 0

while count < 5:
    print(f"カウント: {count}")
    count += 1

print("ループ終了")
```

注意： 無限ループに注意！必ず終了条件を設定

関数の定義

再利用可能なコードブロック

```
def greet(name):  
    """挨拶をする関数"""  
    message = f"こんにちは、{name}さん！"  
    return message  
  
if __name__ == "__main__":  
    result = greet("太郎")  
    print(result)  # こんにちは、太郎さん！  
  
    result = greet("花子")  
    print(result)  # こんにちは、花子さん！
```


文字列操作

基本的な操作

```
text = "Hello, Python!"  
# 文字列の長さ  
print(len(text)) # 14  
# 大文字・小文字変換  
print(text.upper()) # HELLO, PYTHON!  
print(text.lower()) # hello, python!  
# 分割  
words = text.split(",")  
print(words) # ['Hello', ' Python!']  
# 置換  
new_text = text.replace("Hello", "Hi")  
print(new_text) # Hi, Python!
```

正規表現

文字列パターンマッチング

```
import re

text = "2018年の出来事について、2018th yearと表現されます。"

# 数字を抽出
numbers = re.findall(r"\d+", text)
print(numbers)  # ['2018', '2018']

# 大文字で始まる単語を抽出
words = re.findall(r"[A-Z][a-z]+", text)
print(words)
```

パターン	意味
<code>\d</code>	数字
<code>\w</code>	文字（英数字とアンダースコア）
<code>\s</code>	空白文字
<code>+</code>	1回以上の繰り返し
<code>*</code>	0回以上の繰り返し
<code>[A-Z]</code>	大文字
<code>[a-z]</code>	小文字

正規表現：実践例

文字列の検索と置換

```
import re

text = "電話番号: 03-1234-5678, 090-9876-5432"

# 電話番号を検索
phones = re.findall(r"\d{2,4}-\d{4}-\d{4}", text)
print(phones)  # ['03-1234-5678', '090-9876-5432']

# 括弧内の文字を削除
text2 = "Python (2025) is great!"
clean_text = re.sub(r"\([^)]+\)\s*", "", text2)
print(clean_text)  # Python is great!
```

ファイル操作

ファイルの読み書き

```
# ファイルへの書き込み
with open("sample.txt", "w") as f:
    f.write("Hello, File!\n")
    f.write("Pythonファイル操作\n")
```

```
# ファイルからの読み込み
with open("sample.txt", "r") as f:
    content = f.read()
    print(content)
```

ポイント： `with` 文を使うと自動的にファイルがクローズされる

モジュールとインポート

既存のライブラリを活用

```
# 標準ライブラリのインポート
import math
import datetime

# 数学関数
print(math.sqrt(16))    # 4.0
print(math.pi)         # 3.141592...

# 日付と時刻
now = datetime.datetime.now()
print(now)
```

よく使うライブラリ

- **NumPy** - 数値計算
- **Pandas** - データ分析
- **Matplotlib** - グラフ描画
- **Scikit-learn** - 機械学習
- **TensorFlow/PyTorch** - ディープラーニング
- **Requests** - HTTP通信
- **BeautifulSoup** - Webスクレイピング
- **Flask/Django** - Webアプリ開発

エラーと例外処理

try-except文

```
try:
    # エラーが発生する可能性のあるコード
    result = 10 / 0
except ZeroDivisionError:
    # エラーが発生した場合の処理
    print("ゼロで割ることはできません")
except Exception as e:
    # その他のエラー
    print(f"エラーが発生しました: {e}")
finally:
    # 必ず実行される処理
    print("処理を終了します")
```


デバッグのコツ

よくあるエラーと対処法

1. インデントエラー

- タブとスペースの混在に注意

2. 変数名の間違い

- typo（タイプミス）に注意

3. 型エラー

- データ型の不一致に注意

デバッグ方法：

- `print()` で変数の値を確認
- エラーメッセージをよく読む
- 一行ずつ実行して確認

ベストプラクティス

良いコードを書くために

1. 可読性を重視

- わかりやすい変数名
- 適切なコメント

2. 関数を活用

- コードの再利用
- テストしやすい

3. エラーハンドリング

- 例外処理を適切に実装

4. PEP8に従う

- 統一されたコーディングスタイル

学習リソース

さらに学ぶために

公式ドキュメント：

- Python公式: <https://docs.python.org/ja/3/>
- PEP8: <https://pep8-jp.readthedocs.io/ja/latest/>

学習サイト：

- オンラインチュートリアル
- コーディング練習サイト
- YouTubeの解説動画

まとめ

Pythonの基礎を学びました

1. ✓ Pythonの特徴と実行方法
2. ✓ 基本的なデータ型と操作
3. ✓ 制御構造 (if, for, while)
4. ✓ 関数の定義と使用
5. ✓ 文字列操作と正規表現
6. ✓ ファイル操作とモジュール